



Introduction to Version Control Systems

Git, GitHub & Collaboration

Version Control Workshop

Software Development Fundamentals

June 15, 2026

Outline

What is Version Control?

Definition

A **Version Control System (VCS)** tracks and manages changes to files over time, maintaining a detailed history of every modification.

Centralized VCS (CVCS)

- Single central server stores all files and history
- Developers commit/pull directly to the server
- Examples: *Apache Subversion (SVN)*, *CVS*

Distributed VCS (DVCS)

- Every collaborator holds a **full copy** of the project including its history
- Works offline; more resilient
- Examples: *Git*, *Mercurial*, *Darcs*



Git

- Free, open-source DVCS
- Created by **Linus Torvalds** in 2005
- Handles projects of any size with speed
- Allows simultaneous multi-developer work without overwriting changes
- Runs **locally** on your machine



GitHub

- Online platform for hosting Git repositories in the **cloud**
- Enables collaboration, code review, and issue tracking
- Git and GitHub are complementary tools

“Git tracks your history locally — GitHub shares it with the world.”

Installing Git — Windows

Step 1: Download

- Visit git-scm.com and download the **64-bit Windows Setup .exe**

Step 2: Install

- Run the installer, accept the license, and click *Next*

Step 3: Configure PATH

- Select *"Git from the command line and also from 3rd-party software"*

Step 4: Verify

```
git --version
```

Step 5: Set credentials

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Installing Git — Linux

Open a terminal and run:

```
sudo apt update && sudo apt upgrade  
sudo apt install git
```

Confirm the installation:

```
git --version
```

Configure your identity:

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Tip

These credentials are attached to every commit you make, so set them correctly from the start.

Initializing a Repository

Create a new Git repository in your project folder:

```
git init
```

After initializing, explore with these commands:

Command	Description
<code>git status</code>	Current state of working directory & staging area
<code>git log</code>	Chronological list of all commits
<code>git --help</code>	Display help for Git commands

The Core Workflow

Staging & Committing

`git add .` Moves all changes to the **staging area** (index) — a draft board between your working directory and permanent history.

`git commit -m "..."` Saves a **permanent snapshot** of staged changes. Use a meaningful message to describe what changed.

```
git add .  
git commit -m "Describe what changed"  
git push
```

Best Practice

Commit often with clear messages. You can always restore your project to any snapshot.

Method 1: GitHub CLI (gh)

Install on Windows

```
winget install GitHub.cli    # using winget  
scoop install gh            # using Scoop
```

Authenticate

```
gh auth login
```

Create & push a new repo in one command

```
gh repo create YOUR_PROJECT_NAME \  
  --public --source=. --remote=origin --push
```

Note

Replace `--public` with `--private` if you do not want the repository visible to everyone.

Method 2: Clone an Existing Repository

- 1 Log in to github.com
- 2 Click **New** on the Repositories tab
- 3 Fill in name, description, and visibility, then create
- 4 Clone the repository locally:

```
git clone https://github.com/username/repository.git
```

Add your files, then push:

```
git add .  
git commit -m "Initial Commit"  
git push --set-upstream origin master
```

Method 3: Remote Add (Existing Local Project)

Already have a local project? Link it to a new empty GitHub repo:

```
git init
git remote add origin YOUR_REPOSITORY_URL
git add .
git commit -m "Initial Commit"
git push --set-upstream origin master
```

Tip

YOUR_REPOSITORY_URL looks like:

<https://github.com/username/repository.git>

Name Match

Your local project folder and GitHub repository should share the same name for clarity.

Summary

Key Concepts

- VCS = change tracking over time
- DVCS (Git) vs CVCS (SVN)
- Git runs locally; GitHub hosts in the cloud

Essential Commands

- `git init` — start a repo
- `git add .` — stage changes
- `git commit -m` — save snapshot
- `git push` — upload to GitHub

3 Ways to Connect to GitHub

- 1 **gh CLI** — fastest for new projects
- 2 **Clone** — start from GitHub
- 3 **Remote add** — link an existing local project

Next Steps

Explore README.md files, .gitignore, branching (`git branch`), and pull requests on GitHub.



Thank You!

Start tracking your code today.



git-scm.com



github.com



`git --help`